CarryPay: from sandbox to product

Version 1.0 · 15 October 2026 · companion to TECHNICAL.md and SECURITY.md

CarryPay today runs against a simulated world: the banks, card network, hospitals and universities are sandbox programs. This document shows that the simulation sits behind a few narrow seams, what replaces each one in production, who would operate the product legally, and how a pilot with one bank would run. The short version: **the agent, the safety gateway, the rules engine, the passkey approvals and the log stay as they are; the simulated rails are swapped for a bank's real ones, and the bank, not CarryPay, holds and moves the money.**

Regulatory points below were checked on 15 October 2026 against the sources listed at the end. Like the rules in CarryPay's rule packs, they are a starting point for a bank's compliance team, not legal advice.

1. What is real today, and what is simulated

Part	Today	Status
AI agent	DeepSeek V4 Pro over the real API, 17 tools, event-driven loop	**Real**
Safety gateway, rules engine	Code that checks every tool call; rule packs with sources and status	**Real**
Privacy firewall	Names, IDs, card and passport numbers and medical details masked before the model	**Real**
Family approval	WebAuthn passkeys (fingerprint or face) on real phones, signature over the exact payment	**Real**
Activity log and proof	SHA-256 hash chain; head hash anchored in real transactions on the Conflux eSpace testnet	**Real** (testnet)
Reading bills, voice	PDF text extraction, vision model for photos, speech-to-text	**Real**
Hosting	HTTPS server, private sandbox per visitor	**Real**
Banks and payments	SWIFT transfers, returns, settlement in `src/server/sim/`	Simulated
Card network	Authorisations, declines, refunds	Simulated
Hospitals and universities	Rule-based auto-responders, fictional institutions	Simulated
Email	Delivered inside the app	Simulated
Verified institution registry	A fixed list in code	Simulated
FX rates and fees	Illustrative sandbox rates	Simulated
Customer identity (KYC)	None: sandbox people	Not built

2. The swap points

Everything simulated is reached through a few functions. Each row is a replacement of one module; the agent and gateway above it don't change.

Simulated part	Where it is today	Production replacement
Bank transfer	`submitPayment()` in `src/server/sim/engine.ts`, called by `runtime.ts` only after the family's passkey approval	The partner bank's payment API: a transfer from the family's own account (SWIFT MT103 or ISO 20022 pacs.008). The bank officer view (`/bank/officer`) already builds a draft MT103 from each payment.
Payment status	`setPaymentStatus()` in the same file (submitted, completed, returned, refunded)	Status events from the bank, e.g. the Swift gpi tracker by UETR, mapped onto the same statuses. The agent already handles returned transfers, delays and refunds through them.
Card payments	The card branch of `submitPayment()`	The issuer's or processor's authorisation API, with the card network's own authentication.
Email to and from institutions	`onOutboundMail()` and `sendMail()` in `src/server/sim/`	A real mailbox per bank (SMTP/IMAP or a mail API). Inbound mail is checked for SPF, DKIM and DMARC before the agent sees it. The existing rule that the agent may only write to an institution's verified address (control C4) stays.
Verified institutions	`INSTITUTIONS` in `src/domain/institutions.ts`; `verifyInstitution()` for uploaded bills	A registry the bank maintains: the institution, its billing address and its account, confirmed before first use, for example with Swift's Payment Pre-validation (beneficiary account verification) plus a call-back. The gateway's check (`CARRYPAY.VERIFIED_BENEFICIARY`, `CARRYPAY.VERIFIED_INSTITUTION`) reads the registry instead of the list.
FX rates and fees	`src/domain/market.ts`	The bank's live rates and fee schedule, and the card scheme's rates. The engine prices routes the same way.
Regulations	`src/domain/rules/packs/*.json`	The same files, reviewed by the bank's compliance team: every rule marked *reported* or *assumed* is confirmed or corrected, changes go through review and the test suite.
Forms	`prepare_documents` drafts (e.g. SBP Appendix V-68)	The bank's own e-forms, filled from the same data and signed with the approval.
Identity	Sandbox people	The bank's customer login and KYC; the passkey is registered to the bank's customer.
Storage	JSON files in a data folder	A database and an append-only log store, with backups.
Model	DeepSeek API	A model the bank approves. DeepSeek's V4 weights are published under the MIT licence, so a bank can run the same model in its own data centre, and family data never leaves the bank.
Public anchoring	Conflux eSpace testnet	A chain the bank chooses: a public chain, or a consortium chain such as FISCO BCOS (open source, built for finance by the Shenzhen consortium WeBank helped found). The anchor contract is about 20 lines and ports to any EVM chain.

The same seams serve a bank's own assistant: CarryPay is already an MCP server (TECHNICAL.md §4.9), so a bank's chatbot or a staff assistant can call CarryPay's tools and inherit its gateway, rules and passkey approvals, instead of re-implementing them.

What doesn't change: the agent loop and its 17 tools, the safety gateway, the rules engine and route pricing, the privacy firewall, passkey verification, the hash-chained log, and the evaluation and red-team harnesses, which become release tests (§5).

3. Who operates it: a bank, not CarryPay

CarryPay never holds or moves money itself. It is software that a licensed bank deploys for its customers: the bank does the KYC, holds the accounts, executes the transfers and stays responsible to its regulator. This is how it can be legal in both countries without CarryPay becoming a payment company.

Pakistan. Foreign-exchange remittances for medical treatment and education are made by banks authorised to deal in foreign exchange (Authorized Dealers), under the State Bank of Pakistan's foreign-exchange rules that CarryPay's Pakistan pack encodes. A partner bank would run CarryPay inside that existing process. The State Bank also runs a **regulatory sandbox**: guidelines issued in May 2025, a first cohort shortlisted in January 2026 (themes: inward remittances, open banking, remote merchant onboarding), live testing for up to six months. The State Bank has made clear that completing the sandbox is not approval to launch: a product still needs its authorisation. Outward medical and education payments were not a first-cohort theme, so the realistic route is through a partner bank, or a future cohort.

China. Personal foreign-exchange purchases go through banks under SAFE's rules (the annual quota, the purpose declaration, the ban on splitting across relatives), which CarryPay's China pack encodes and a bank already enforces. Three further sets of rules matter:

- **Generative AI.** The *Interim Measures for the Management of Generative AI Services* (in force since 15 August 2023) cover generative AI services offered to the public in China; a public-facing service with certain attributes needs a security assessment and an algorithm filing. A bank offering CarryPay to customers would check whether its deployment falls under them, along with its own regulators' rules.
- **Personal data.** The March 2024 rules on cross-border data flows exempt transfers that are necessary to perform a contract with the individual, with payments given as an example. CarryPay's design goes further: with a self-hosted model and data kept in the bank, there is no transfer to the model provider at all.
- **The bank's own regulators.** Sector rules for banks and financial technology apply on top; the bank's compliance team owns this.

4. Security for production

The sandbox already has the core controls (SECURITY.md). Production adds:

- **Keys in hardware.** The anchor wallet's key, session secrets and API keys move into the bank's HSM or key management service; no key in files.
- **Independent testing.** A penetration test and an external red team before launch, and an AI red team before every model or prompt change.
- **The bank stays in control.** A bank-level emergency stop beside the family's; limits per customer and per day set by the bank; every payment package visible to the bank's FX officer, as the officer view already shows.
- **Clear liability.** The AI never authorises a payment: the family's passkey does, and the bank's usual controls apply after it. Disputes use the bank's existing process, with the hash-chained log as the record of what the agent did and why.
- **Monitoring.** Alerts on blocked tool calls, fraud flags and failed anchors; a weekly sample of cases reviewed by a person.

5. Proving it stays safe: release gates

The tools we built to evaluate the prototype become the gates every release must pass:

1. `npm test`: 88 tests, including 18 attacks on the safety gateway, must pass.
2. `node scripts/eval.mjs`: every scenario run live several times; the release is blocked if any safety check fails (payment to an unverified account, rule broken, family not warned).
3. `npm run redteam:llm`: the 20 live-model attacks, with **0 executed** required; new attacks are added as they are seen in the wild.
4. Log verification and on-chain anchors checked for every pilot case.

6. A pilot with one bank

Phase	Scope	Done when
0. Prepare	One partner bank, one corridor (Pakistan → Thailand hospital, or China → UK tuition). Compliance review of the rule pack; registry of the first hospitals or universities with verified accounts; model hosted in the bank's test environment; bank APIs connected at the swap points in §2.	Every rule in the pack confirmed by the bank; all release gates pass on the bank's test systems.
1. Staff-assisted pilot	Real cases. CarryPay prepares each payment; the family approves with a passkey; the bank's FX officer reviews the package before it is sent.	An agreed number of real cases with **zero payments to unverified accounts**; time from invoice to payment, cost saved against paying by card, and officer review time measured.
2. Self-service	Families use the app directly; the officer reviews only flagged cases. A second corridor and more institutions.	Pilot measures hold without the officer's review of every case.

7. What it costs to run

- **AI.** Measured on the live model: $0.05 to $0.23 per case at DeepSeek's peak list prices (EVAL.md); the nine-day hospital case is the most expensive. A self-hosted model replaces this with the bank's own server costs.
- **Chain.** One small transaction per milestone (1 to 6 per case in our evidence runs), on any EVM chain; on a consortium chain, no fee to a public network.
- **What the family gains**, from the engine on sandbox rates: in the Bangkok case, paying the deposit by bank transfer instead of card saved about PKR 44,000, and the bill check recovered THB 14,000 charged twice. Real savings depend on the bank's real rates and fees.

8. Open questions for a partner bank

- Which corridor first, and which of its institutions can be verified quickly?
- Is the bank's payment API able to report status events (or Swift gpi updates) to CarryPay?
- Which model may the bank use: a hosted provider or a self-hosted open-weight model?
- Who signs off rule-pack changes, and how often are they reviewed?
- What does the bank want the officer to see before a payment leaves?

Sources (checked 15 October 2026)

- State Bank of Pakistan regulatory sandbox, first cohort and its themes: Profit / Pakistan Today, 7 Jan 2026, The Asian Banker; sandbox is not approval to launch: ProPakistani, 16 Sep 2026
- China's generative AI measures: China Briefing, DLA Piper
- China's cross-border data provisions (March 2024): Faegre Drinker, Hunton
- DeepSeek V4 open weights under the MIT licence: deepseek-ai on Hugging Face (check each model's LICENSE file)
- FISCO BCOS: official documentation
- Swift Payment Pre-validation: swift.com
- Pakistan's and China's foreign-exchange rules: the sources in `src/domain/rules/packs/pk-sbp.json` and `cn-safe.json`, each with its status